Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D,
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.utils import plot_model
```

```
2024-04-27 20:44:28.335628: I tensorflow/core/util/port.cc:113] oneDNN custom operat
ions are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the enviro
nment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2024-04-27 20:44:28.384068: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.c
c:9261] Unable to register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
2024-04-27 20:44:28.384113: E external/local_xla/xla/stream_executor/cuda/cuda_fft.c
c:607] Unable to register cuFFT factory: Attempting to register factory for plugin c
uFFT when one has already been registered
2024-04-27 20:44:28.385673: E external/local_xla/xla/stream_executor/cuda/cuda_blas.
cc:1515] Unable to register cuBLAS factory: Attempting to register factory for plugi
n cuBLAS when one has already been registered
2024-04-27 20:44:28.398285: I tensorflow/core/platform/cpu_feature_guard.cc:182] Thi
s TensorFlow binary is optimized to use available CPU instructions in performance-cr
itical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operati
ons, rebuild TensorFlow with the appropriate compiler flags.
2024-04-27 20:44:29.680313: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38]
TF-TRT Warning: Could not find TensorRT
```

Import Dataset

In [ ]:
```
df = pd.read_csv('Dataset/Bank_Loan_Granting.csv')
df.head()
```

Out[ ]:

	ID	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	Personal Loan	S
0	1	25	1	49	91107	4	1/60	1	0	0	
1	2	45	19	34	90089	3	1/50	1	0	0	
2	3	39	15	11	94720	1	1/00	1	0	0	
3	4	35	9	100	94112	1	2/70	2	0	0	
4	5	35	8	45	91330	4	1/00	2	0	0	

Fix Data Loading Errors and Deleting useless variable

In [ ]:
```
df.drop(columns=['ID'], axis=1, inplace=True)
```

Dropped because it is not useful for the model and unrelated to the target variable.

In [ ]:
```
# Change all / in CCAvg into . and change into double
df['CCAvg'] = df['CCAvg'].replace(to_replace=r'/', value='.', regex=True)
df['CCAvg'] = df['CCAvg'].astype(float)
```

Is done because the data inside in unrecognizable and seems to have a typo which is fixed by changing the '/' into '.' and making it a float

In [ ]:
```
df.head()
```

Out[ ]:

	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	Personal Loan	Secu Acc
0	25	1	49	91107	4	1.6	1	0	0	
1	45	19	34	90089	3	1.5	1	0	0	
2	39	15	11	94720	1	1.0	1	0	0	
3	35	9	100	94112	1	2.7	2	0	0	
4	35	8	45	91330	4	1.0	2	0	0	

Data Exploration

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Age                 5000 non-null   int64  
 1   Experience          5000 non-null   int64  
 2   Income              5000 non-null   int64  
 3   ZIP Code            5000 non-null   int64  
 4   Family              5000 non-null   int64  
 5   CCAvg               5000 non-null   float64
 6   Education           5000 non-null   int64  
 7   Mortgage            5000 non-null   int64  
 8   Personal Loan       5000 non-null   int64  
 9   Securities Account  5000 non-null   int64  
 10  CD Account          5000 non-null   int64  
 11  Online              5000 non-null   int64  
 12  CreditCard          5000 non-null   int64  
dtypes: float64(1), int64(12)
memory usage: 507.9 KB
```

In [ ]:
```
df.isnull().sum()
```

Out[ ]:
```
Age                   0
Experience            0
Income                0
ZIP Code              0
Family                0
CCAvg                 0
Education             0
Mortgage              0
Personal Loan         0
Securities Account    0
CD Account            0
Online                0
CreditCard            0
dtype: int64
```

The dataset has no NULL or NA values

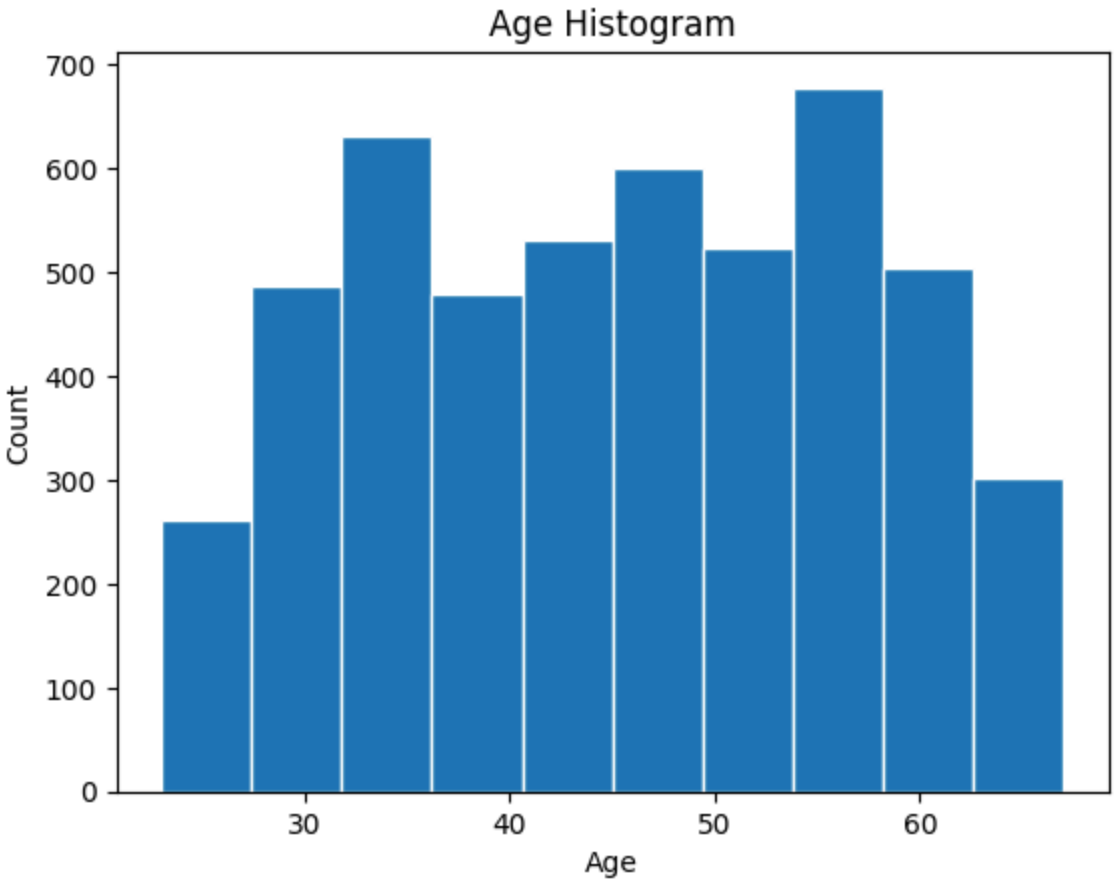
Check for Imbalance

Customer Information 'Age', Experience, ZIP Code, Family, Education, Income'

In [ ]:
```
customerInfo = ['Age', 'Experience', 'ZIP Code', 'Family', 'Education', 'Income']
```

In [ ]:
```
plt.hist(df['Age'], edgecolor='white')
plt.title('Age Histogram')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```

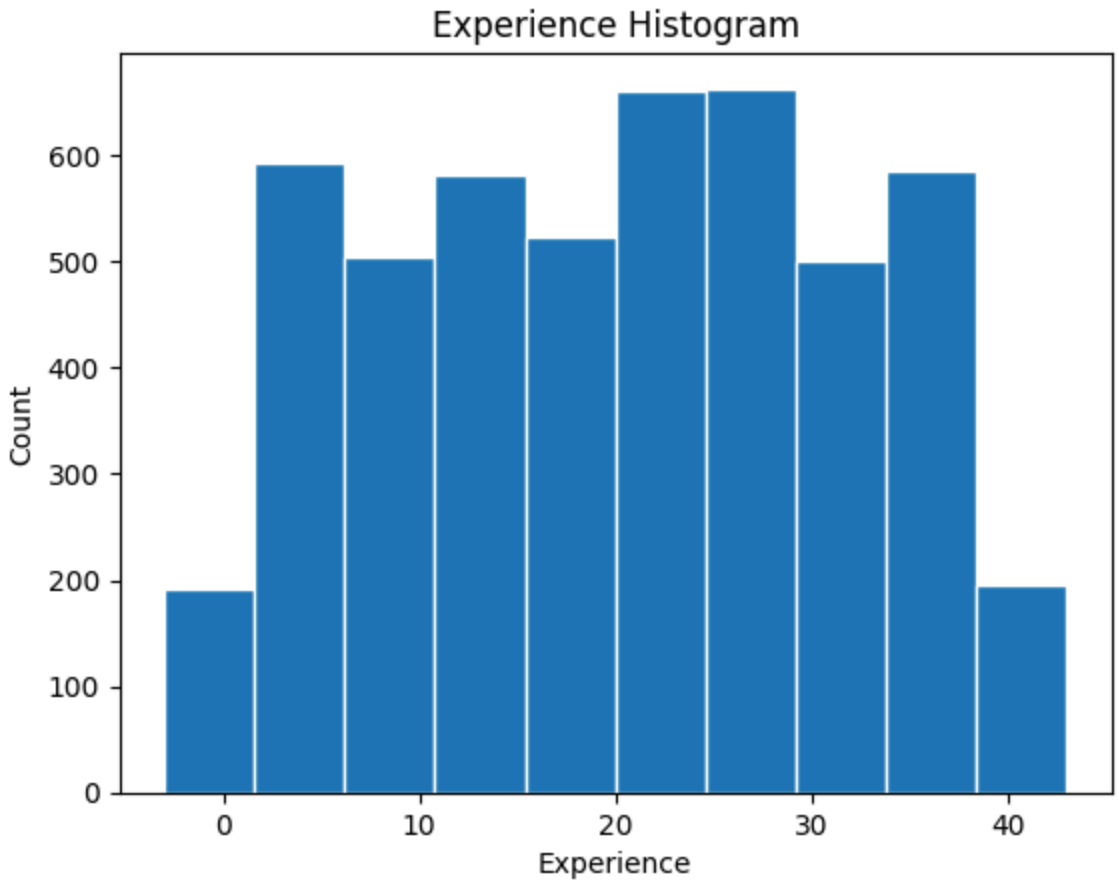
```
df['Age'].describe()
```



Out[ ]:
```
count    5000.000000
mean       45.338400
std        11.463166
min        23.000000
25%        35.000000
50%        45.000000
75%        55.000000
max        67.000000
Name: Age, dtype: float64
```

The Age distribution is similar to a multimodal distribution with the average being 45.339 and standart deviation being 11.463. With the peaks at approximately 35 and 55 Years.

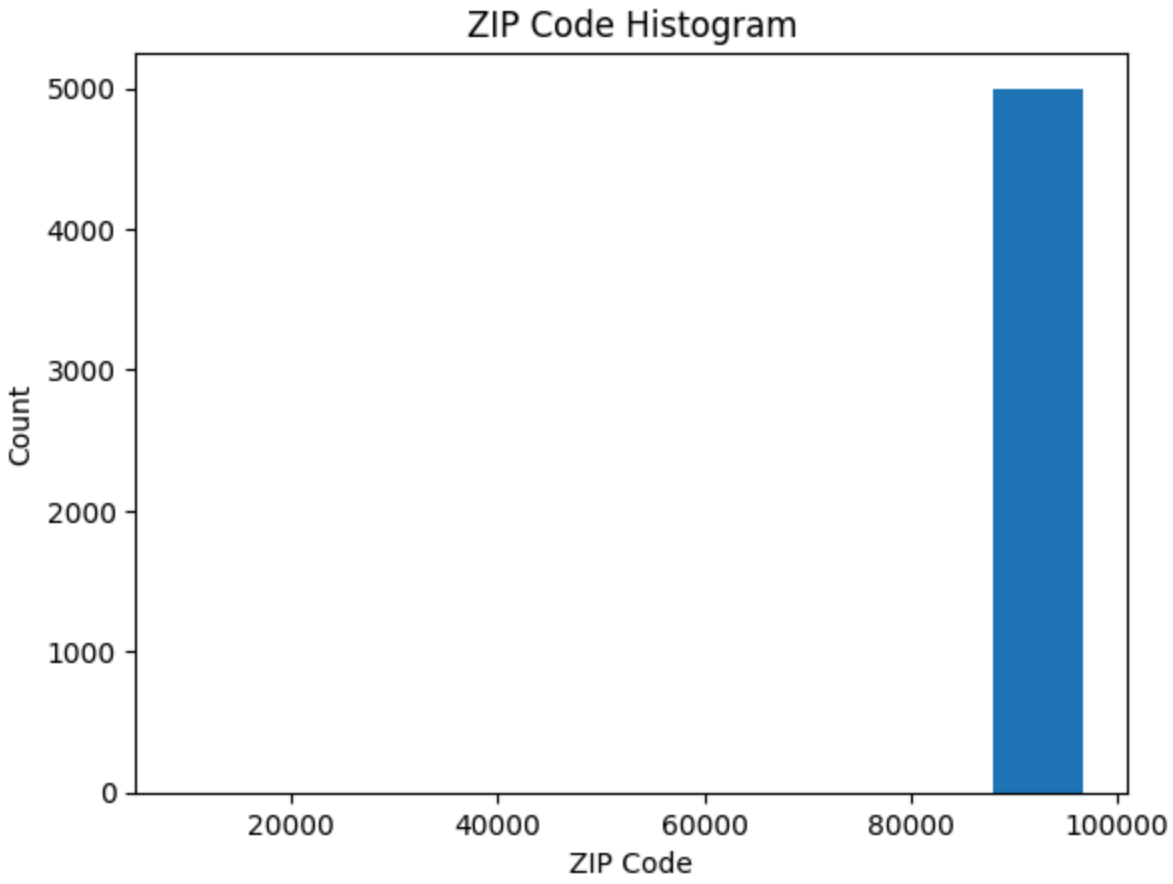
In [ ]:
```
plt.hist(df['Experience'], edgecolor='white')
plt.title('Experience Histogram')
plt.xlabel('Experience')
plt.ylabel('Count')
plt.show()

df['Experience'].describe()
```



Out[ ]:
```
count    5000.000000
mean       20.104600
std        11.467954
min        -3.000000
25%        10.000000
50%        20.000000
75%        30.000000
max        43.000000
Name: Experience, dtype: float64
```

The Experience distribution is similar to a multimodal distribution with the average being 20.105 and standart deviation being 11.468. With the peaks at approximately at 20 to 30 Years.

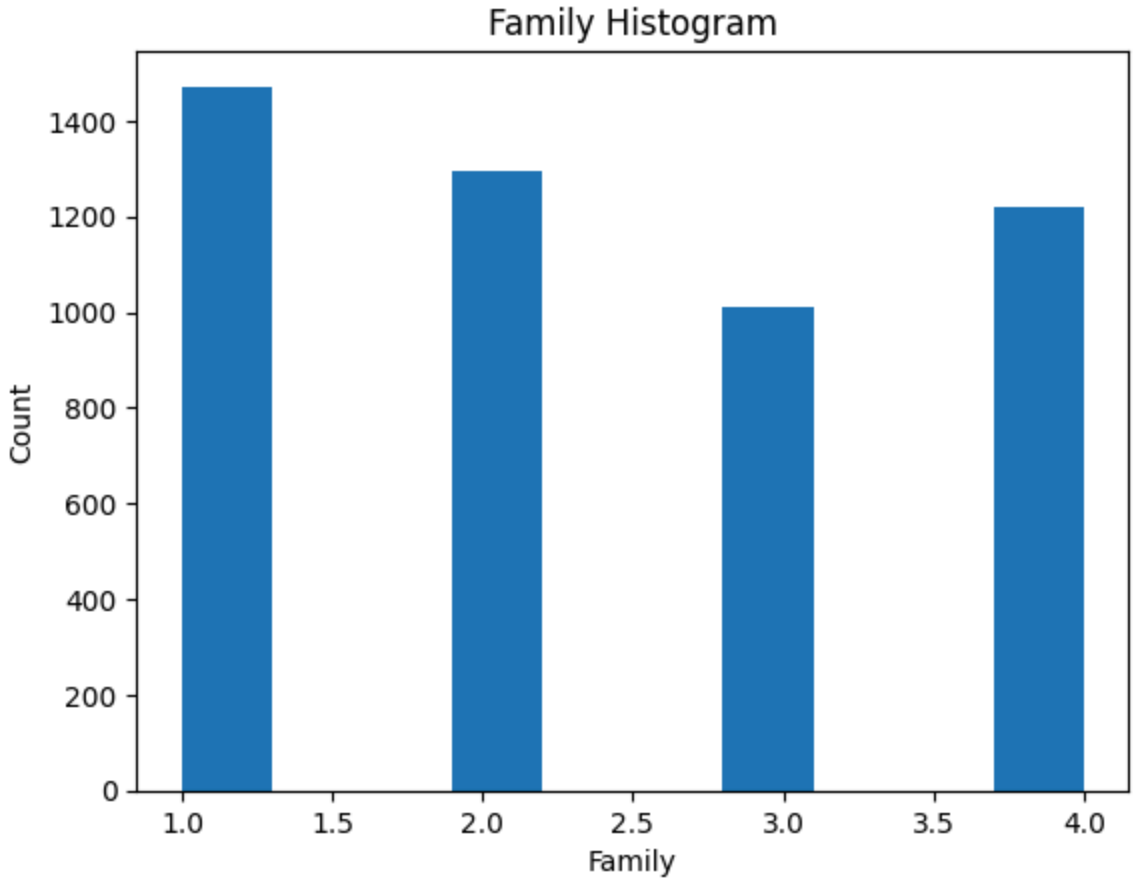
In [ ]:
```
plt.hist(df['ZIP Code'])
plt.title('ZIP Code Histogram')
plt.xlabel('ZIP Code')
plt.ylabel('Count')
plt.show()

df['ZIP Code'].describe()
```



Out[ ]:
```
count     5000.000000
mean     93152.503000
std       2121.852197
min       9307.000000
25%      91911.000000
50%      93437.000000
75%      94608.000000
max      96651.000000
Name: ZIP Code, dtype: float64
```

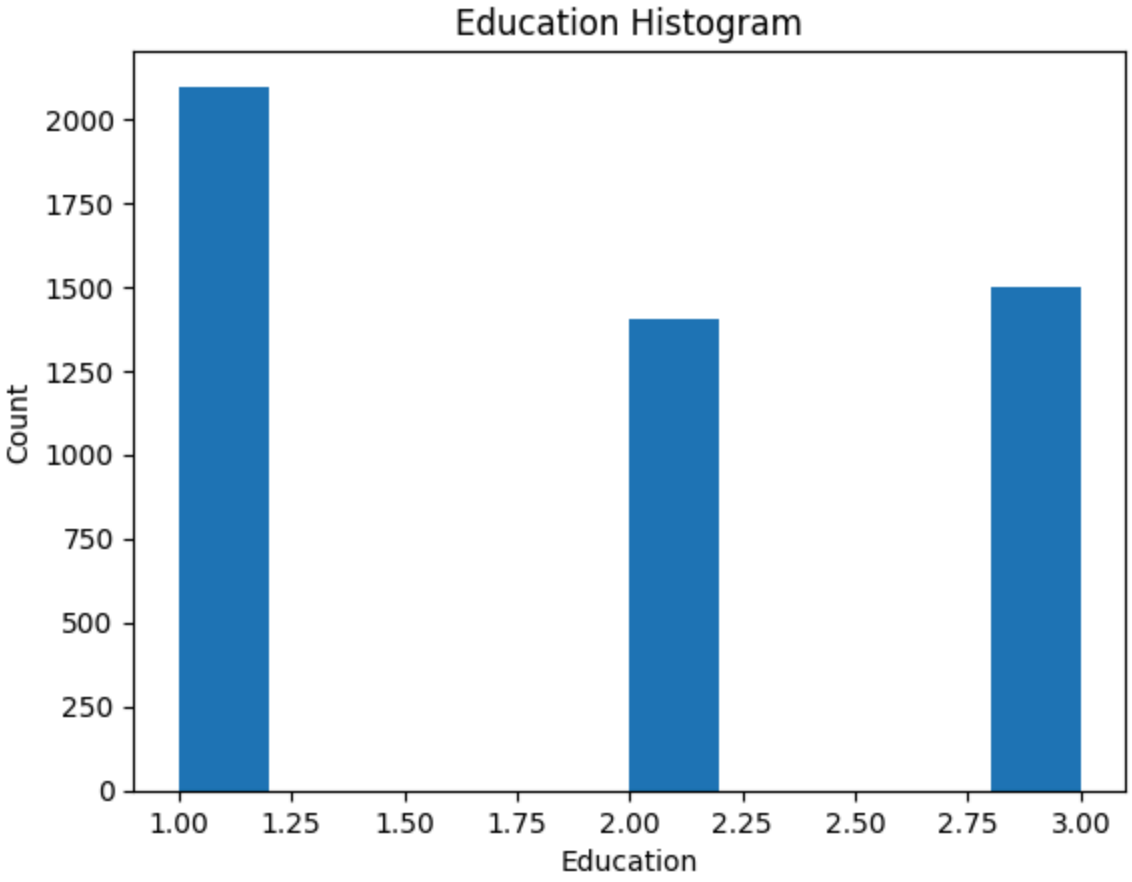
The ZIP Code has no clear distribution with the mean being 93152.4 and the standart deviation being 2121.85. Which could be dropped because it can cause underfitting.

In [ ]:
```
plt.hist(df['Family'])
plt.title('Family Histogram')
plt.xlabel('Family')
plt.ylabel('Count')
plt.show()
```



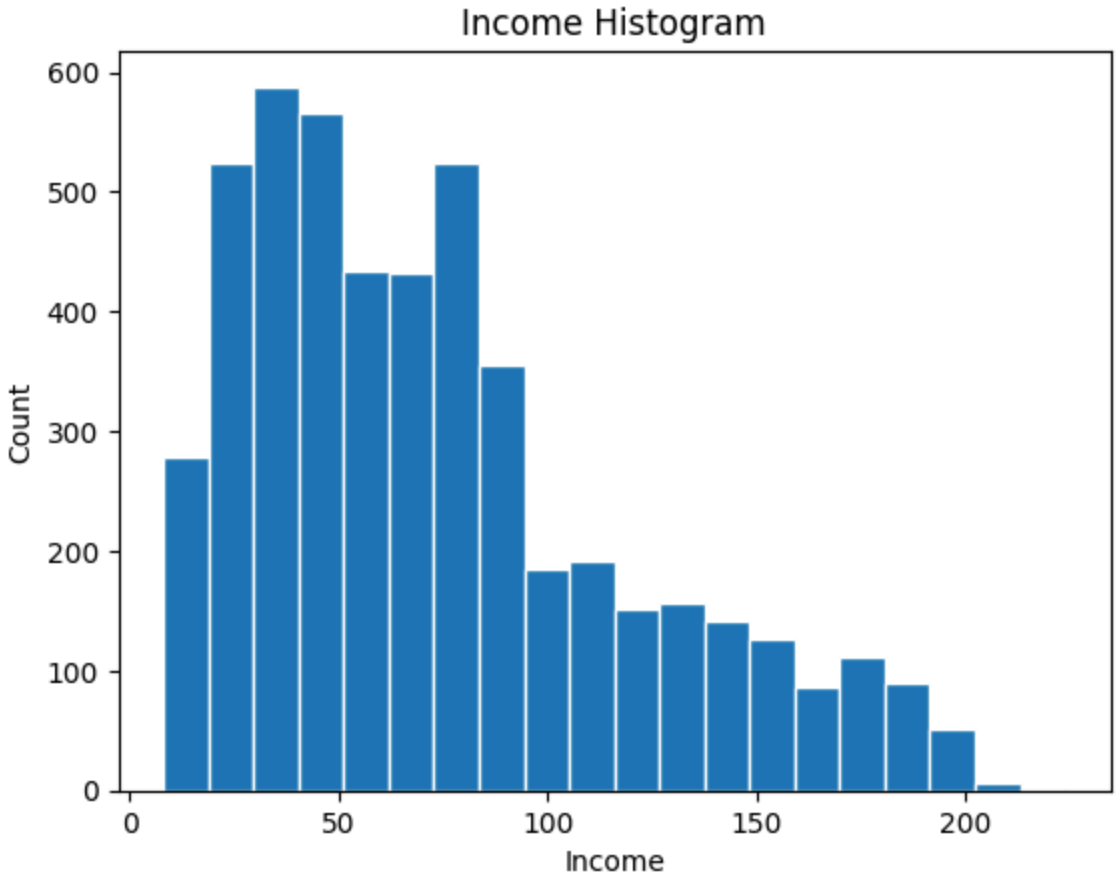
The Family is evenly distributed between 1 to 4 with the majority being 1 Family Members, Followed with 2, 4, and 3 Family members subsequently.

In [ ]:
```
plt.hist(df['Education'])
plt.title('Education Histogram')
plt.xlabel('Education')
plt.ylabel('Count')
plt.show()
```



The Education is evenly distributed between 1 to 3 with the majority being 1 (Undergraduate) followed with 3 (Advanced) and 2 (Graduate) Degree subsequently.

In [ ]:
```
plt.hist(df['Income'], edgecolor='white', bins=20)
plt.title('Income Histogram')
plt.xlabel('Income')
plt.ylabel('Count')
plt.show()

df['Income'].describe()
```

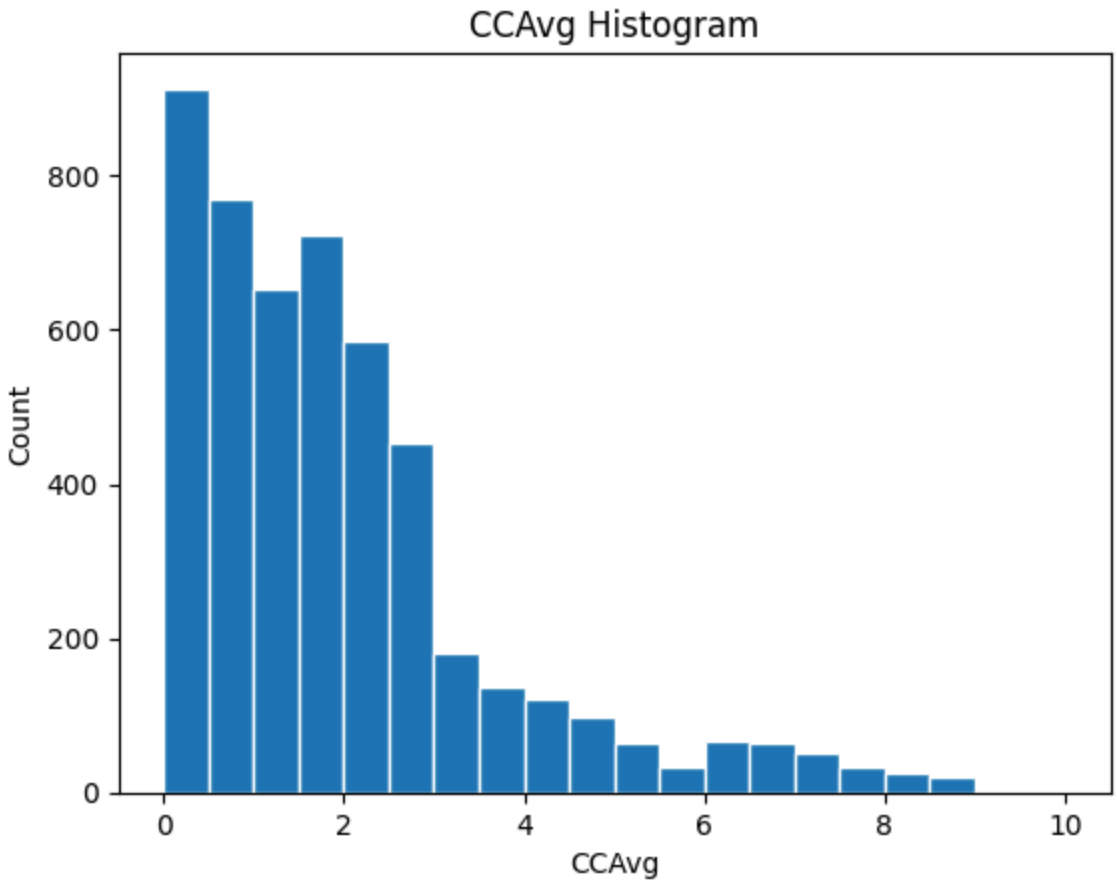


Out[ ]:
```
count    5000.000000
mean       73.774200
std        46.033729
min         8.000000
25%        39.000000
50%        64.000000
75%        98.000000
max       224.000000
Name: Income, dtype: float64
```

The Income Histogram is Right skewed with the peak being approximately around 0 to 50k but with a long tail of customer making high incomes. With it having Average of 73.774 and Standart Deviation of 46.034. This may an indication to do normalization to the data.

Customer Behavior (CCAvg, Mortgage, Persona' Loan, Securities Account, CD Account, Online, CreditCard)

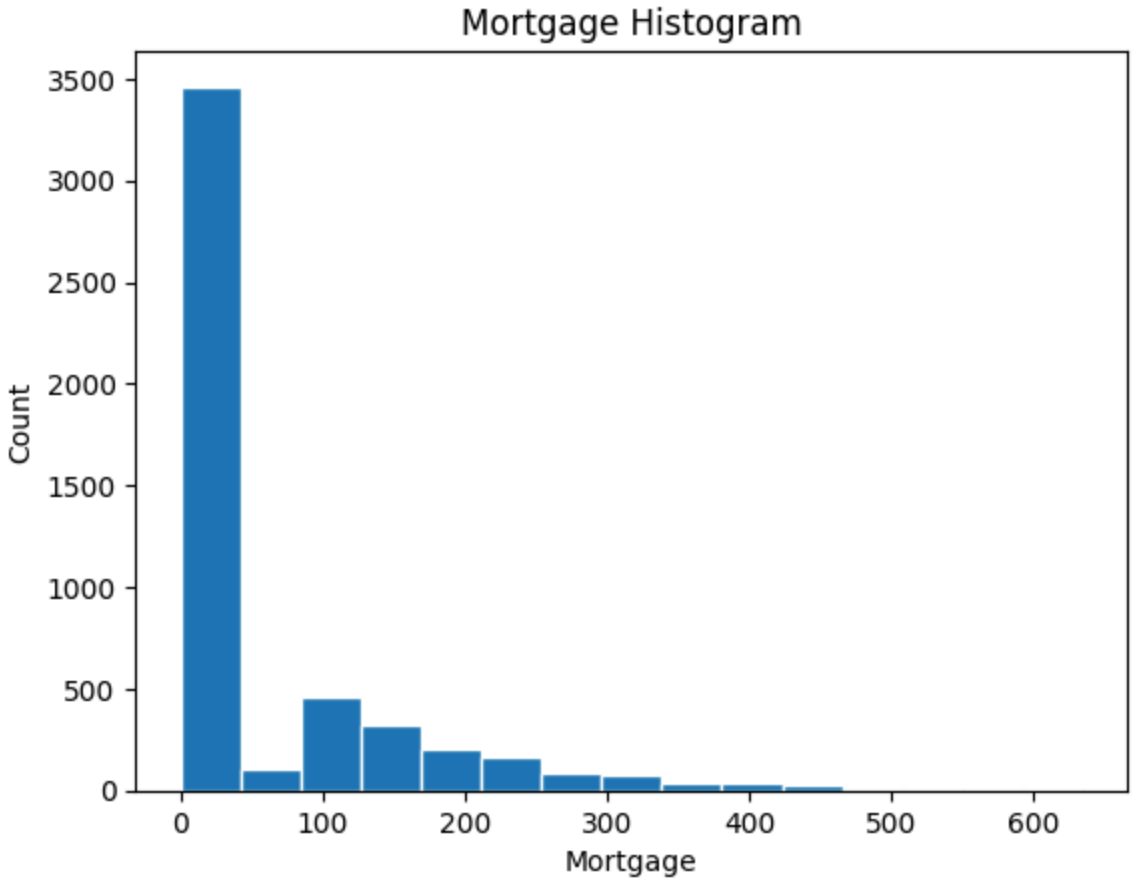
In [ ]:
```
plt.hist(df['CCAvg'], edgecolor='white', bins=20)
plt.title('CCAvg Histogram')
plt.xlabel('CCAvg')
plt.ylabel('Count')
plt.show()

df['CCAvg'].describe()
```



Out[ ]:
```
count    5000.000000
mean        1.937938
std         1.747659
min         0.000000
25%         0.700000
50%         1.500000
75%         2.500000
max        10.000000
Name: CCAvg, dtype: float64
```

The Credit Card Average Spending Histogram is skewed to the right with the peak being 0 or no spending. With the average being 1.937 and standart deviation being 1.747. This may an indication to do normalization to the data.

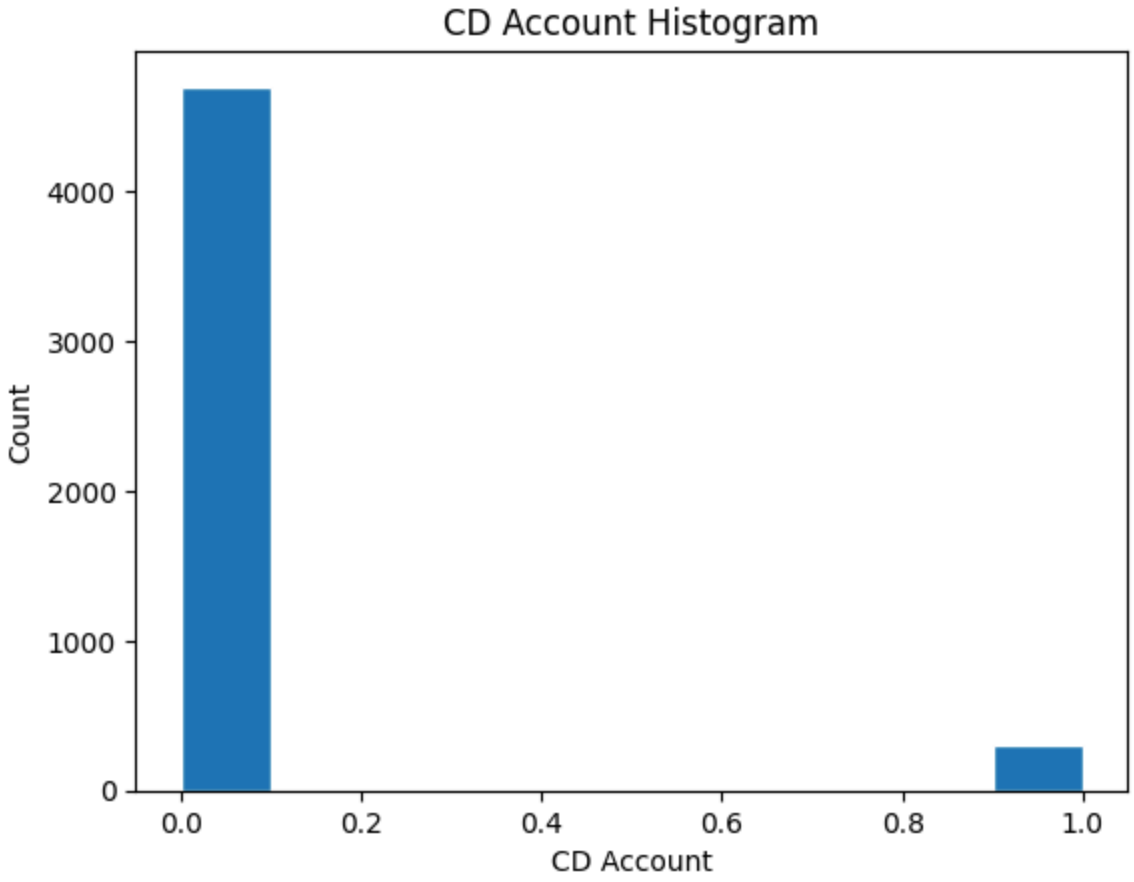
In [ ]:
```
plt.hist(df['Mortgage'], edgecolor='white', bins = 15)
plt.title('Mortgage Histogram')
plt.xlabel('Mortgage')
plt.ylabel('Count')
plt.show()

df['Mortgage'].describe()
```



Out[ ]:
```
count    5000.000000
mean       56.498800
std       101.713802
min         0.000000
25%         0.000000
50%         0.000000
75%       101.000000
max       635.000000
Name: Mortgage, dtype: float64
```

The Morgage Spending is around 0 and 100k on average with a long tail of higher mortgage spending. With the average being 56.5 and standart deviation being 101.7. This may an indication to do normalization to the data.

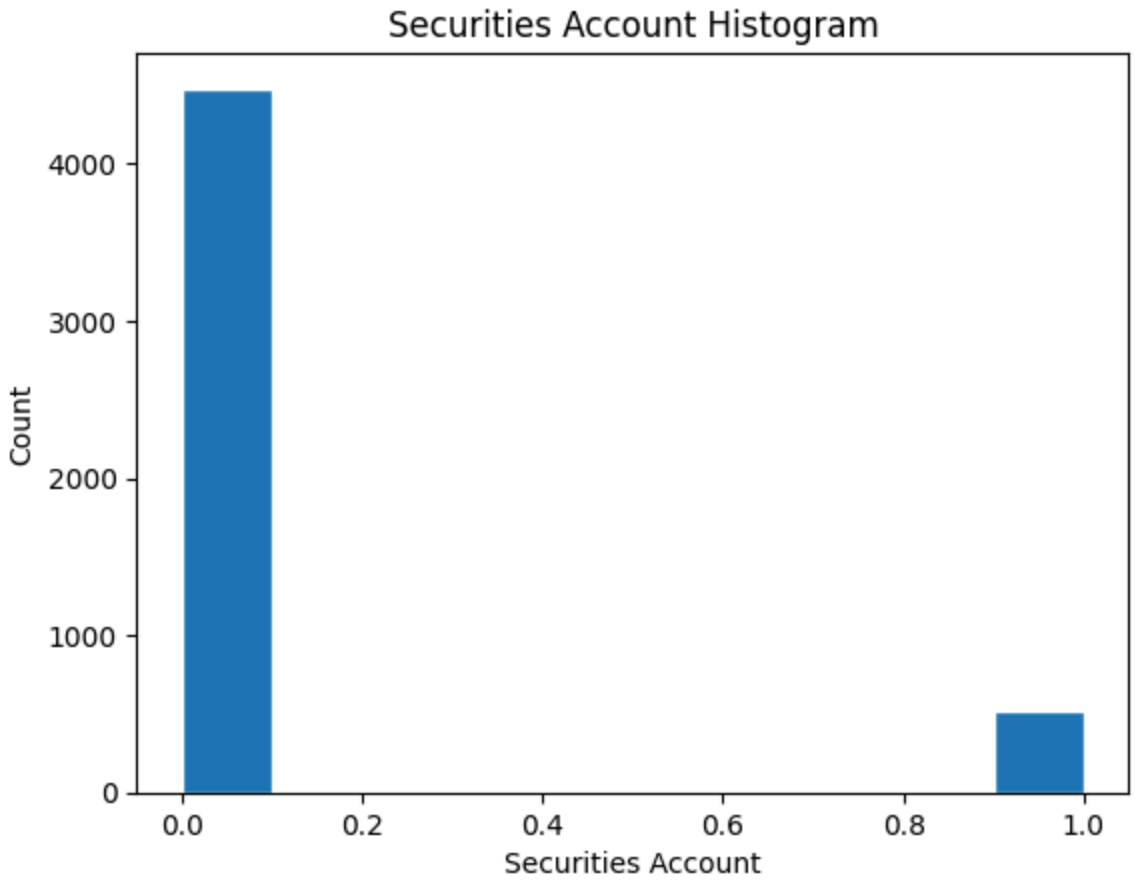
In [ ]:
```
plt.hist(df['CD Account'], edgecolor='white')
plt.title('CD Account Histogram')
plt.xlabel('CD Account')
plt.ylabel('Count')
plt.show()

df['CD Account'].value_counts(normalize=True)
```



Out[ ]:
```
CD Account
0    0.9396
1    0.0604
Name: proportion, dtype: float64
```

The CD Account Histogram shows that most customer doesn't have a Deposit(CD) Account with the proportion being 94% no Deposit account and 6% with Deposit Account

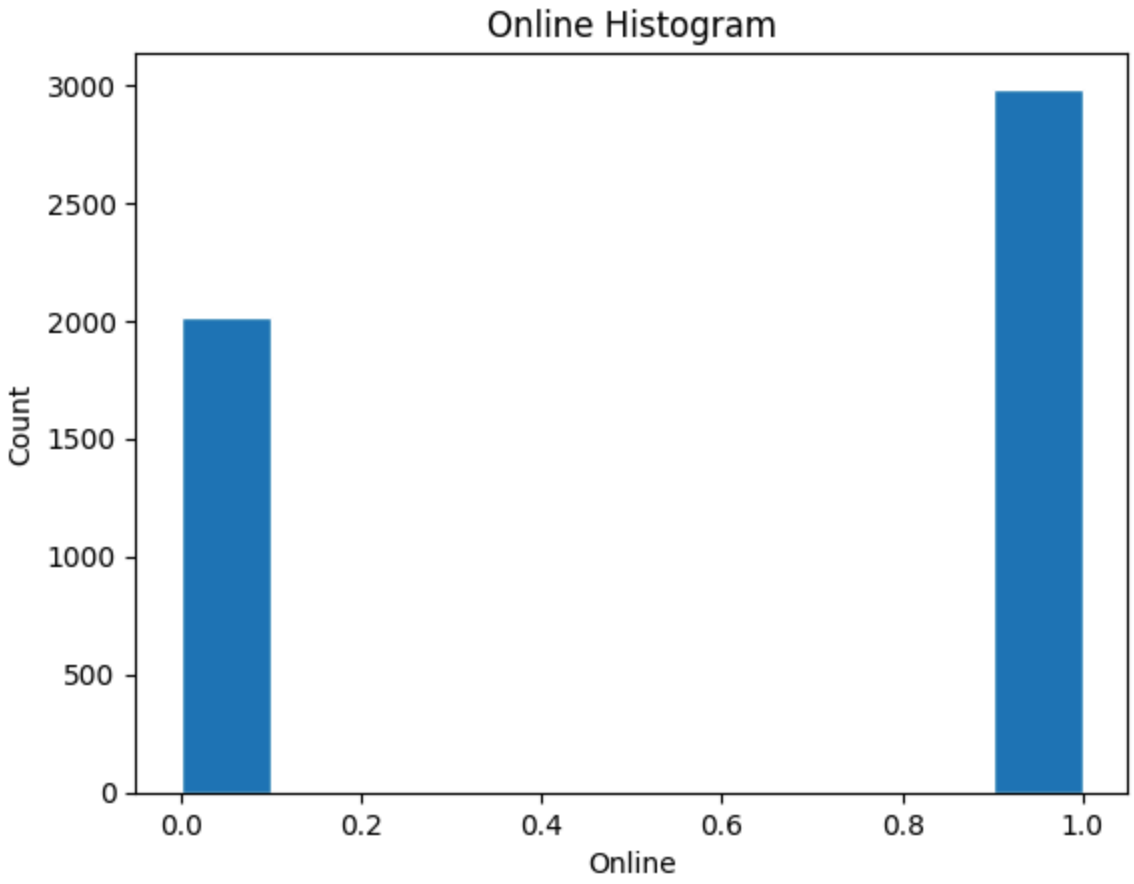
In [ ]:
```
plt.hist(df['Securities Account'], edgecolor='white')
plt.title('Securities Account Histogram')
plt.xlabel('Securities Account')
plt.ylabel('Count')
plt.show()

df['Securities Account'].value_counts(normalize=True)
```



Out[ ]:
```
Securities Account
0    0.8956
1    0.1044
Name: proportion, dtype: float64
```

The Securities account Histogram shows that most customer doesn't have a Securities Account with the proportion being 90% no Securities account and 10% with Securities Account

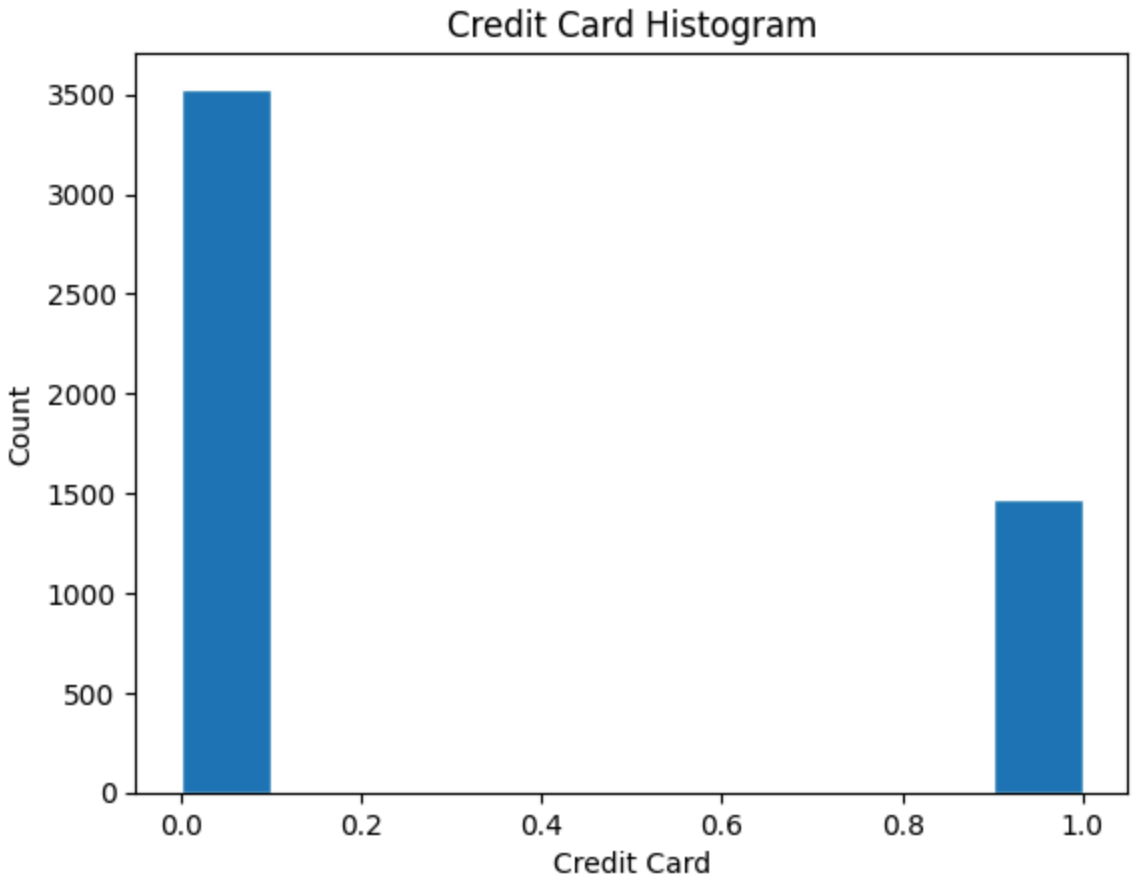
In [ ]:
```
plt.hist(df['Online'], edgecolor='white')
plt.title('Online Histogram')
plt.xlabel('Online')
plt.ylabel('Count')
plt.show()

df['Online'].value_counts(normalize=True)
```



Out[ ]:
```
Online
1    0.5968
0    0.4032
Name: proportion, dtype: float64
```

The Online Account Histogram shows that most customer have an Online Account with the proportion being 60% with Online Account and 40% without Online Account

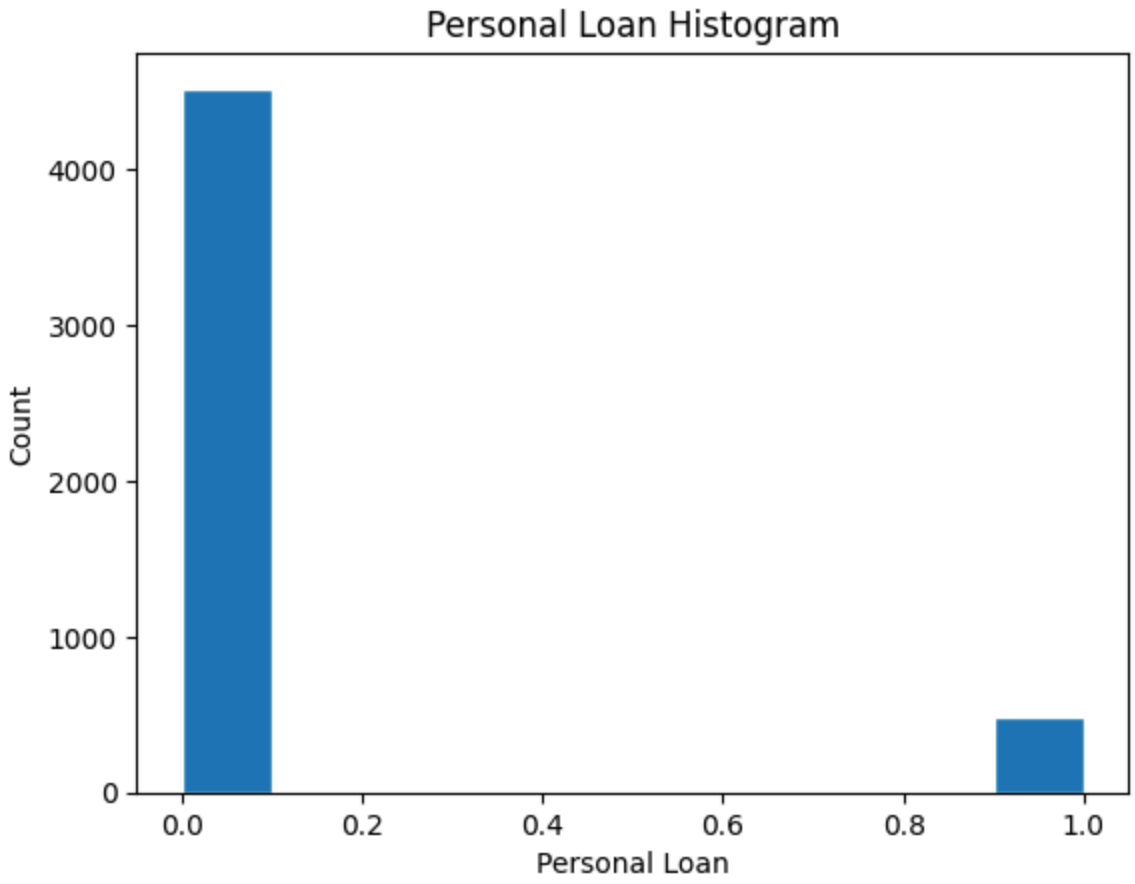
In [ ]:
```
plt.hist(df['CreditCard'], edgecolor='white')
plt.title('Credit Card Histogram')
plt.xlabel('Credit Card')
plt.ylabel('Count')
plt.show()

df['CreditCard'].value_counts(normalize=True)
```



Out[ ]:
```
CreditCard
0    0.706
1    0.294
Name: proportion, dtype: float64
```

The Credit Card Histogram shows that most customer doesn't have a Credit Card with the proportion being 70% no Credit Card and 30% with Credit Card

In [ ]:
```
plt.hist(df['Personal Loan'], edgecolor='white')
plt.title('Personal Loan Histogram')
plt.xlabel('Personal Loan')
plt.ylabel('Count')
plt.show()

df['Personal Loan'].value_counts(normalize=True)
```



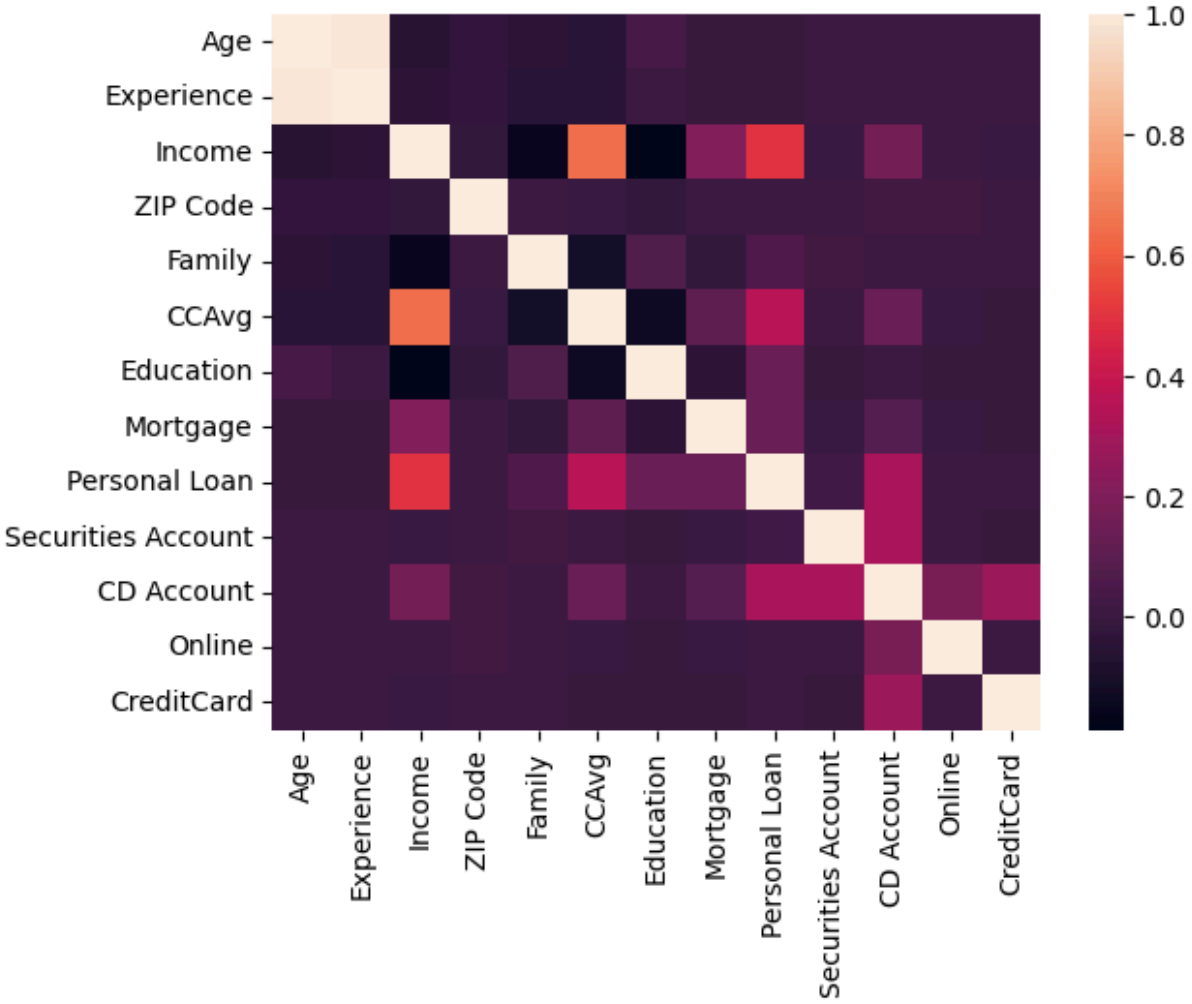
Out[ ]:
```
Personal Loan
0    0.904
1    0.096
Name: proportion, dtype: float64
```

The Personal Loan Histogram shows that most customer doesn't have a Personal Loan with the proportion being 90% no Personal Loan and 10% with Personal Loan. This indicates that the dataset is imbalanced.

Check Correlation

In [ ]:
```
sns.heatmap(df.corr())
```

Out[ ]:
```
<Axes: >
```



From the heatmap we can see that

- Age and Experience is highly correlated (Could lead to multicolinearity)
- Personal Loan is Highly corelated to Income, CCAvg, and CD Account and somewhat correlated with Mortgage, Education, and Number of Family Members with it having little to no corelation with CreditCard, Online, Securities Account, ZIP Code, Experience, and Age

1.c Problem in the dataset

The Problem in the dataset is:

- The Existence of Multicolinearity between Age and Experience
- The Existence of Imbalance in the Personal Loan Target Variable (90% No Personal Loan and 10% Yes Personal Loan)
- The Existence of Unnecessary Variable (ZIP Code)
- The Existence of Outliers in the Income, CCAvg, and Mortgage Variable shown by the long tail in the histogram

- Some data is uncorelated to the target variable (CreditCard, Online, Securities Account, ZIP Code, Experience, and Age)

1.a Preprocessing

What we will do:

- Drop Variable that has little to no correlation to Personal Loan to avoid underfitting.
- Drop Variable that's highly correlated to each other to avoid multicolinearity.
- Spliting Dataset (train = 80%, valid = 20%, test = 20%)
- Normalize the data to avoid the model being biased to outlier, resulting in underperformance or underfitting

Dropping uncorrelated variable

In [ ]:
```
newDf = df.drop(columns=['CreditCard', 'Online', 'Securities Account', 'ZIP Code',
```

In [ ]:
```
newDf.head()
```

Out[ ]:

	Income	Family	CCAvg	Education	Mortgage	Personal Loan	CD Account
0	49	4	1.6	1	0	0	0
1	34	3	1.5	1	0	0	0
2	11	1	1.0	1	0	0	0
3	100	1	2.7	2	0	0	0
4	45	4	1.0	2	0	0	0

Spliting Data

In [ ]:
```
X, y = newDf.drop(columns=['Personal Loan'], axis=1), newDf['Personal Loan']

X_train, X_val_test, y_train, y_val_test = train_test_split(X, y, test_size=0.2, ra
X_val, X_test, y_val, y_test = train_test_split(X_val_test, y_val_test, test_size=0
```

Normalization

In [ ]:
```
scaler = StandardScaler()
```

Train dataset

In [ ]:
```
scaled_x_train = X_train.drop(columns=['Family', 'Education', 'CD Account'])
scaled_x_train = scaler.fit_transform(scaled_x_train)

X_train = pd.concat([pd.DataFrame(scaled_x_train), X_train.drop(columns=['Mortgage'
```

In [ ]:
```
X_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4000 entries, 0 to 3999
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   0          4000 non-null   float64
 1   1          4000 non-null   float64
 2   2          4000 non-null   float64
 3   Family     4000 non-null   int64  
 4   CCAvg      4000 non-null   float64
 5   Education  4000 non-null   int64  
dtypes: float64(4), int64(2)
memory usage: 187.6 KB
```

Valid dataset

In [ ]:
```
scaled_x_val = X_val.drop(columns=['Family', 'Education', 'CD Account'])
scaled_x_val = scaler.transform(scaled_x_val)

X_val = pd.concat([pd.DataFrame(scaled_x_val), X_val.drop(columns=['Mortgage', 'CD
```

Test dataset

In [ ]:
```
scaled_x_test = X_test.drop(columns=['Family', 'Education', 'CD Account'])
scaled_x_test = scaler.transform(scaled_x_test)

X_test = pd.concat([pd.DataFrame(scaled_x_test), X_test.drop(columns=['Mortgage', '
```

Create Tensor Dataset

In [ ]:
```
train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(32)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(32)
val_ds = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(32)
```

```
2024-04-27 20:44:36.355398: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.421395: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.421691: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.427152: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.427280: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.427321: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.644807: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.644908: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.644919: I tensorflow/core/common_runtime/gpu/gpu_device.cc:2022]
Could not identify NUMA node of platform GPU id 0, defaulting to 0.  Your kernel may
not have been built with NUMA support.
2024-04-27 20:44:36.644956: I external/local_xla/xla/stream_executor/cuda/cuda_execu
tor.cc:887] could not open file to read NUMA node: /sys/bus/pci/devices/0000:2e:00.
0/numa_node
Your kernel may have been built without NUMA support.
2024-04-27 20:44:36.644975: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1929]
Created device /job:localhost/replica:0/task:0/device:GPU:0 with 1347 MB memory:  ->
device: 0, name: NVIDIA GeForce MX450, pci bus id: 0000:2e:00.0, compute capability:
7.5
```

1.d

Modelling

In [ ]:
```
inputs = tf.keras.Input(shape=(6,))
dense1 = Dense(12, activation='relu')(inputs)
dense2 = Dense(12, activation='relu')(dense1)
outputs = Dense(1, activation='sigmoid')(dense2)
```

```
model1 = Model(inputs=inputs, outputs=outputs)
model1.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 6)]               0

 dense (Dense)               (None, 12)                84

 dense_1 (Dense)             (None, 12)                156

 dense_2 (Dense)             (None, 1)                 13

=================================================================
Total params: 253 (1012.00 Byte)
Trainable params: 253 (1012.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [ ]:
```
model1.compile(loss='binary_crossentropy', optimizer=tf.keras.optimizers.Adam(), me
```

In [ ]:
```
my_callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, mode='auto') #
]

history = model1.fit(train_ds, validation_data=val_ds, epochs=100, callbacks=my_cal
```

```
Epoch 1/100
2024-04-27 20:44:38.847973: I external/local_xla/xla/service/service.cc:168] XLA ser
vice 0x7f3eb90a3990 initialized for platform CUDA (this does not guarantee that XLA
will be used). Devices:
2024-04-27 20:44:38.848078: I external/local_xla/xla/service/service.cc:176]   Strea
mExecutor device (0): NVIDIA GeForce MX450, Compute Capability 7.5
2024-04-27 20:44:38.894122: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_ut
il.cc:269] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIREC
TORY` to enable.
2024-04-27 20:44:38.948708: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.c
c:454] Loaded cuDNN version 8904
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
RR
I0000 00:00:1714225479.124943   33156 device_compiler.h:186] Compiled cluster using
XLA!  This line is logged at most once for the lifetime of the process.
```

```
125/125 [==============================] - 4s 12ms/step - loss: 0.4905 - precision:
0.1193 - val_loss: 0.3930 - val_precision: 1.0000
Epoch 2/100
125/125 [==============================] - 1s 11ms/step - loss: 0.2746 - precision:
0.9286 - val_loss: 0.2176 - val_precision: 1.0000
Epoch 3/100
125/125 [==============================] - 1s 10ms/step - loss: 0.1983 - precision:
0.9897 - val_loss: 0.1551 - val_precision: 0.9688
Epoch 4/100
125/125 [==============================] - 1s 10ms/step - loss: 0.1616 - precision:
0.8802 - val_loss: 0.1155 - val_precision: 0.9792
Epoch 5/100
125/125 [==============================] - 1s 10ms/step - loss: 0.1360 - precision:
0.9030 - val_loss: 0.0935 - val_precision: 0.9800
Epoch 6/100
125/125 [==============================] - 1s 10ms/step - loss: 0.1191 - precision:
0.9141 - val_loss: 0.0781 - val_precision: 0.9804
Epoch 7/100
125/125 [==============================] - 1s 10ms/step - loss: 0.1068 - precision:
0.9134 - val_loss: 0.0669 - val_precision: 0.9811
Epoch 8/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0985 - precision:
0.9241 - val_loss: 0.0597 - val_precision: 0.9643
Epoch 9/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0922 - precision:
0.9158 - val_loss: 0.0550 - val_precision: 0.9636
Epoch 10/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0881 - precision:
0.9136 - val_loss: 0.0516 - val_precision: 0.9636
Epoch 11/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0852 - precision:
0.9068 - val_loss: 0.0489 - val_precision: 0.9474
Epoch 12/100
125/125 [==============================] - 1s 9ms/step - loss: 0.0831 - precision:
0.9045 - val_loss: 0.0467 - val_precision: 0.9310
Epoch 13/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0812 - precision:
0.9054 - val_loss: 0.0447 - val_precision: 0.9310
Epoch 14/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0798 - precision:
0.9012 - val_loss: 0.0433 - val_precision: 0.9310
Epoch 15/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0788 - precision:
0.8957 - val_loss: 0.0421 - val_precision: 0.9310
Epoch 16/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0780 - precision:
0.8957 - val_loss: 0.0412 - val_precision: 0.9322
Epoch 17/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0773 - precision:
0.8933 - val_loss: 0.0404 - val_precision: 0.9322
Epoch 18/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0767 - precision:
0.8933 - val_loss: 0.0397 - val_precision: 0.9322
Epoch 19/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0762 - precision:
0.8960 - val_loss: 0.0391 - val_precision: 0.9322
```

```
Epoch 20/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0758 - precision:
0.8967 - val_loss: 0.0386 - val_precision: 0.9322
Epoch 21/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0753 - precision:
0.8994 - val_loss: 0.0382 - val_precision: 0.9322
Epoch 22/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0749 - precision:
0.9003 - val_loss: 0.0377 - val_precision: 0.9322
Epoch 23/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0745 - precision:
0.9006 - val_loss: 0.0374 - val_precision: 0.9322
Epoch 24/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0741 - precision:
0.9088 - val_loss: 0.0370 - val_precision: 0.9322
Epoch 25/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0737 - precision:
0.9091 - val_loss: 0.0367 - val_precision: 0.9322
Epoch 26/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0734 - precision:
0.9091 - val_loss: 0.0364 - val_precision: 0.9322
Epoch 27/100
125/125 [==============================] - 2s 15ms/step - loss: 0.0730 - precision:
0.9091 - val_loss: 0.0361 - val_precision: 0.9322
Epoch 28/100
125/125 [==============================] - 2s 15ms/step - loss: 0.0727 - precision:
0.9094 - val_loss: 0.0357 - val_precision: 0.9322
Epoch 29/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0724 - precision:
0.9094 - val_loss: 0.0355 - val_precision: 0.9322
Epoch 30/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0721 - precision:
0.9096 - val_loss: 0.0352 - val_precision: 0.9322
Epoch 31/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0718 - precision:
0.9127 - val_loss: 0.0348 - val_precision: 0.9322
Epoch 32/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0715 - precision:
0.9127 - val_loss: 0.0346 - val_precision: 0.9322
Epoch 33/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0713 - precision:
0.9102 - val_loss: 0.0343 - val_precision: 0.9322
Epoch 34/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0710 - precision:
0.9102 - val_loss: 0.0340 - val_precision: 0.9322
Epoch 35/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0707 - precision:
0.9102 - val_loss: 0.0337 - val_precision: 0.9322
Epoch 36/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0705 - precision:
0.9104 - val_loss: 0.0335 - val_precision: 0.9322
Epoch 37/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0702 - precision:
0.9075 - val_loss: 0.0333 - val_precision: 0.9322
Epoch 38/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0699 - precision:
```

```
0.9077 - val_loss: 0.0331 - val_precision: 0.9322
Epoch 39/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0695 - precision:
0.9083 - val_loss: 0.0326 - val_precision: 0.9322
Epoch 40/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0692 - precision:
0.9134 - val_loss: 0.0323 - val_precision: 0.9322
Epoch 41/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0688 - precision:
0.9162 - val_loss: 0.0320 - val_precision: 0.9483
Epoch 42/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0685 - precision:
0.9162 - val_loss: 0.0318 - val_precision: 0.9483
Epoch 43/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0683 - precision:
0.9164 - val_loss: 0.0316 - val_precision: 0.9492
Epoch 44/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0680 - precision:
0.9167 - val_loss: 0.0314 - val_precision: 0.9492
Epoch 45/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0678 - precision:
0.9194 - val_loss: 0.0312 - val_precision: 0.9492
Epoch 46/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0675 - precision:
0.9199 - val_loss: 0.0310 - val_precision: 0.9492
Epoch 47/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0673 - precision:
0.9199 - val_loss: 0.0308 - val_precision: 0.9492
Epoch 48/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0671 - precision:
0.9201 - val_loss: 0.0307 - val_precision: 0.9492
Epoch 49/100
125/125 [==============================] - 2s 15ms/step - loss: 0.0668 - precision:
0.9201 - val_loss: 0.0306 - val_precision: 0.9492
Epoch 50/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0666 - precision:
0.9201 - val_loss: 0.0304 - val_precision: 0.9492
Epoch 51/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0664 - precision:
0.9201 - val_loss: 0.0303 - val_precision: 0.9492
Epoch 52/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0662 - precision:
0.9201 - val_loss: 0.0302 - val_precision: 0.9492
Epoch 53/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0660 - precision:
0.9228 - val_loss: 0.0301 - val_precision: 0.9492
Epoch 54/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0658 - precision:
0.9228 - val_loss: 0.0301 - val_precision: 0.9492
Epoch 55/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0657 - precision:
0.9228 - val_loss: 0.0299 - val_precision: 0.9492
Epoch 56/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0655 - precision:
0.9228 - val_loss: 0.0298 - val_precision: 0.9492
Epoch 57/100
```

```
125/125 [==============================] - 1s 11ms/step - loss: 0.0654 - precision:
0.9231 - val_loss: 0.0297 - val_precision: 0.9492
Epoch 58/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0652 - precision:
0.9231 - val_loss: 0.0296 - val_precision: 0.9492
Epoch 59/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0650 - precision:
0.9231 - val_loss: 0.0293 - val_precision: 0.9492
Epoch 60/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0649 - precision:
0.9231 - val_loss: 0.0293 - val_precision: 0.9492
Epoch 61/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0647 - precision:
0.9228 - val_loss: 0.0291 - val_precision: 0.9492
Epoch 62/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0645 - precision:
0.9233 - val_loss: 0.0290 - val_precision: 0.9492
Epoch 63/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0644 - precision:
0.9233 - val_loss: 0.0289 - val_precision: 0.9492
Epoch 64/100
125/125 [==============================] - 2s 15ms/step - loss: 0.0642 - precision:
0.9231 - val_loss: 0.0287 - val_precision: 0.9492
Epoch 65/100
125/125 [==============================] - 2s 17ms/step - loss: 0.0642 - precision:
0.9233 - val_loss: 0.0287 - val_precision: 0.9492
Epoch 66/100
125/125 [==============================] - 2s 18ms/step - loss: 0.0640 - precision:
0.9231 - val_loss: 0.0286 - val_precision: 0.9492
Epoch 67/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0638 - precision:
0.9260 - val_loss: 0.0283 - val_precision: 0.9492
Epoch 68/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0637 - precision:
0.9235 - val_loss: 0.0283 - val_precision: 0.9492
Epoch 69/100
125/125 [==============================] - 2s 17ms/step - loss: 0.0636 - precision:
0.9260 - val_loss: 0.0282 - val_precision: 0.9492
Epoch 70/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0634 - precision:
0.9260 - val_loss: 0.0281 - val_precision: 0.9492
Epoch 71/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0633 - precision:
0.9292 - val_loss: 0.0279 - val_precision: 0.9492
Epoch 72/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0631 - precision:
0.9292 - val_loss: 0.0279 - val_precision: 0.9492
Epoch 73/100
125/125 [==============================] - 2s 15ms/step - loss: 0.0630 - precision:
0.9292 - val_loss: 0.0277 - val_precision: 0.9492
Epoch 74/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0629 - precision:
0.9292 - val_loss: 0.0276 - val_precision: 0.9483
Epoch 75/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0628 - precision:
0.9292 - val_loss: 0.0275 - val_precision: 0.9483
```

```
Epoch 76/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0627 - precision:
0.9292 - val_loss: 0.0272 - val_precision: 0.9483
Epoch 77/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0625 - precision:
0.9292 - val_loss: 0.0272 - val_precision: 0.9483
Epoch 78/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0624 - precision:
0.9292 - val_loss: 0.0271 - val_precision: 0.9483
Epoch 79/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0622 - precision:
0.9292 - val_loss: 0.0269 - val_precision: 0.9483
Epoch 80/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0620 - precision:
0.9292 - val_loss: 0.0267 - val_precision: 0.9483
Epoch 81/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0619 - precision:
0.9292 - val_loss: 0.0267 - val_precision: 0.9483
Epoch 82/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0618 - precision:
0.9292 - val_loss: 0.0266 - val_precision: 0.9483
Epoch 83/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0617 - precision:
0.9290 - val_loss: 0.0266 - val_precision: 0.9483
Epoch 84/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0615 - precision:
0.9292 - val_loss: 0.0265 - val_precision: 0.9483
Epoch 85/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0613 - precision:
0.9292 - val_loss: 0.0266 - val_precision: 0.9483
Epoch 86/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0611 - precision:
0.9290 - val_loss: 0.0265 - val_precision: 0.9483
Epoch 87/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0609 - precision:
0.9347 - val_loss: 0.0264 - val_precision: 0.9492
Epoch 88/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0606 - precision:
0.9347 - val_loss: 0.0264 - val_precision: 0.9483
Epoch 89/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0604 - precision:
0.9345 - val_loss: 0.0262 - val_precision: 0.9492
Epoch 90/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0602 - precision:
0.9349 - val_loss: 0.0261 - val_precision: 0.9492
Epoch 91/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0600 - precision:
0.9349 - val_loss: 0.0260 - val_precision: 0.9333
Epoch 92/100
125/125 [==============================] - 1s 12ms/step - loss: 0.0598 - precision:
0.9403 - val_loss: 0.0259 - val_precision: 0.9492
Epoch 93/100
125/125 [==============================] - 1s 10ms/step - loss: 0.0596 - precision:
0.9375 - val_loss: 0.0259 - val_precision: 0.9492
Epoch 94/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0595 - precision:
```

```
0.9375 - val_loss: 0.0259 - val_precision: 0.9492
Epoch 95/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0593 - precision:
0.9429 - val_loss: 0.0258 - val_precision: 0.9492
Epoch 96/100
125/125 [==============================] - 2s 14ms/step - loss: 0.0592 - precision:
0.9429 - val_loss: 0.0257 - val_precision: 0.9492
Epoch 97/100
125/125 [==============================] - 2s 12ms/step - loss: 0.0590 - precision:
0.9433 - val_loss: 0.0256 - val_precision: 0.9492
Epoch 98/100
125/125 [==============================] - 2s 13ms/step - loss: 0.0589 - precision:
0.9433 - val_loss: 0.0255 - val_precision: 0.9333
Epoch 99/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0588 - precision:
0.9433 - val_loss: 0.0255 - val_precision: 0.9492
Epoch 100/100
125/125 [==============================] - 1s 11ms/step - loss: 0.0588 - precision:
0.9431 - val_loss: 0.0254 - val_precision: 0.9492
```

Model Evaluation

In [ ]:
```
model1.evaluate(test_ds)
```

```
 7/16 [============>.................] - ETA: 0s - loss: 0.0573 - precision: 0.9412
16/16 [==============================] - 0s 13ms/step - loss: 0.0656 - precision: 0.
9487
```

Out[ ]:
```
[0.06561530381441116, 0.9487179517745972]
```

In [ ]:
```
from sklearn.metrics import classification_report

y_pred = model1.predict(X_test)
predicted = tf.squeeze(y_pred)
predicted = np.array([1 if x >= 0.5 else 0 for x in predicted])

print(classification_report(y_test, predicted))
```

```
16/16 [==============================] - 0s 3ms/step
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       453
           1       0.95      0.79      0.86        47

    accuracy                           0.98       500
   macro avg       0.96      0.89      0.92       500
weighted avg       0.98      0.98      0.97       500
```

1.e

Because the 1st model is already such a good fit for the dataset, we will proceed using the 1st model without any changes to the model and hyperparameter tuning. But it seem that

100 epoch is not enough for the model to train complete ly, we will increase the epoch to 500 to see if the model can improve further.

In [ ]:
```
inputs = tf.keras.Input(shape=(6,))
dense1 = Dense(12, activation='relu')(inputs)
dense2 = Dense(12, activation='relu')(dense1)
outputs = Dense(1, activation='sigmoid')(dense2)

model2 = Model(inputs=inputs, outputs=outputs)
model2.summary()
```

```
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 6)]               0

 dense_3 (Dense)             (None, 12)                84

 dense_4 (Dense)             (None, 12)                156

 dense_5 (Dense)             (None, 1)                 13

=================================================================
Total params: 253 (1012.00 Byte)
Trainable params: 253 (1012.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [ ]:
```
model2.compile(loss='binary_crossentropy', optimizer=tf.keras.optimizers.Adam(), me
```

In [ ]:
```
my_callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, mode='auto') #
]

history = model2.fit(train_ds, validation_data=val_ds, epochs=500, callbacks=my_cal
```

```
Epoch 1/500
125/125 [==============================] - 2s 9ms/step - loss: 0.3677 - precision_1:
0.0000e+00 - val_loss: 0.3186 - val_precision_1: 0.0000e+00
Epoch 2/500
125/125 [==============================] - 1s 8ms/step - loss: 0.2632 - precision_1:
0.5000 - val_loss: 0.2354 - val_precision_1: 0.0000e+00
Epoch 3/500
125/125 [==============================] - 1s 9ms/step - loss: 0.2130 - precision_1:
0.7949 - val_loss: 0.1817 - val_precision_1: 1.0000
Epoch 4/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1817 - precision_1:
0.7867 - val_loss: 0.1495 - val_precision_1: 0.9070
Epoch 5/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1611 - precision_1:
0.8365 - val_loss: 0.1292 - val_precision_1: 0.9038
Epoch 6/500
125/125 [==============================] - 1s 7ms/step - loss: 0.1459 - precision_1:
0.8548 - val_loss: 0.1122 - val_precision_1: 0.9259
Epoch 7/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1327 - precision_1:
0.8677 - val_loss: 0.0984 - val_precision_1: 0.9608
Epoch 8/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1214 - precision_1:
0.9053 - val_loss: 0.0868 - val_precision_1: 0.9808
Epoch 9/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1127 - precision_1:
0.9185 - val_loss: 0.0785 - val_precision_1: 0.9811
Epoch 10/500
125/125 [==============================] - 1s 8ms/step - loss: 0.1058 - precision_1:
0.9206 - val_loss: 0.0725 - val_precision_1: 0.9811
Epoch 11/500
125/125 [==============================] - 1s 9ms/step - loss: 0.1004 - precision_1:
0.9199 - val_loss: 0.0676 - val_precision_1: 0.9818
Epoch 12/500
125/125 [==============================] - 2s 13ms/step - loss: 0.0962 - precision_
1: 0.9153 - val_loss: 0.0639 - val_precision_1: 0.9821
Epoch 13/500
125/125 [==============================] - 2s 12ms/step - loss: 0.0928 - precision_
1: 0.9128 - val_loss: 0.0607 - val_precision_1: 0.9818
Epoch 14/500
125/125 [==============================] - 1s 10ms/step - loss: 0.0900 - precision_
1: 0.9236 - val_loss: 0.0580 - val_precision_1: 0.9818
Epoch 15/500
125/125 [==============================] - 1s 12ms/step - loss: 0.0876 - precision_
1: 0.9274 - val_loss: 0.0560 - val_precision_1: 0.9643
Epoch 16/500
125/125 [==============================] - 1s 12ms/step - loss: 0.0856 - precision_
1: 0.9279 - val_loss: 0.0541 - val_precision_1: 0.9643
Epoch 17/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0839 - precision_1:
0.9253 - val_loss: 0.0524 - val_precision_1: 0.9643
Epoch 18/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0824 - precision_1:
0.9223 - val_loss: 0.0513 - val_precision_1: 0.9643
Epoch 19/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0812 - precision_1:
```

```
0.9260 - val_loss: 0.0494 - val_precision_1: 0.9643
Epoch 20/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0800 - precision_1:
0.9293 - val_loss: 0.0483 - val_precision_1: 0.9818
Epoch 21/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0790 - precision_1:
0.9241 - val_loss: 0.0475 - val_precision_1: 0.9818
Epoch 22/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0781 - precision_1:
0.9211 - val_loss: 0.0463 - val_precision_1: 0.9818
Epoch 23/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0771 - precision_1:
0.9211 - val_loss: 0.0456 - val_precision_1: 0.9818
Epoch 24/500
125/125 [==============================] - 1s 10ms/step - loss: 0.0762 - precision_
1: 0.9185 - val_loss: 0.0446 - val_precision_1: 0.9818
Epoch 25/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0755 - precision_1:
0.9219 - val_loss: 0.0439 - val_precision_1: 0.9818
Epoch 26/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0747 - precision_1:
0.9193 - val_loss: 0.0432 - val_precision_1: 0.9818
Epoch 27/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0740 - precision_1:
0.9172 - val_loss: 0.0424 - val_precision_1: 0.9818
Epoch 28/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0733 - precision_1:
0.9177 - val_loss: 0.0415 - val_precision_1: 0.9818
Epoch 29/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0727 - precision_1:
0.9177 - val_loss: 0.0409 - val_precision_1: 0.9818
Epoch 30/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0721 - precision_1:
0.9179 - val_loss: 0.0402 - val_precision_1: 0.9818
Epoch 31/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0716 - precision_1:
0.9179 - val_loss: 0.0396 - val_precision_1: 0.9815
Epoch 32/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0710 - precision_1:
0.9184 - val_loss: 0.0391 - val_precision_1: 0.9815
Epoch 33/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0704 - precision_1:
0.9184 - val_loss: 0.0388 - val_precision_1: 0.9815
Epoch 34/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0699 - precision_1:
0.9184 - val_loss: 0.0382 - val_precision_1: 0.9815
Epoch 35/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0693 - precision_1:
0.9184 - val_loss: 0.0378 - val_precision_1: 0.9815
Epoch 36/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0688 - precision_1:
0.9187 - val_loss: 0.0374 - val_precision_1: 0.9815
Epoch 37/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0684 - precision_1:
0.9192 - val_loss: 0.0370 - val_precision_1: 0.9815
Epoch 38/500
```

```
125/125 [==============================] - 1s 8ms/step - loss: 0.0678 - precision_1:
0.9192 - val_loss: 0.0365 - val_precision_1: 0.9815
Epoch 39/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0674 - precision_1:
0.9194 - val_loss: 0.0360 - val_precision_1: 0.9815
Epoch 40/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0670 - precision_1:
0.9222 - val_loss: 0.0357 - val_precision_1: 0.9815
Epoch 41/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0666 - precision_1:
0.9226 - val_loss: 0.0353 - val_precision_1: 0.9815
Epoch 42/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0663 - precision_1:
0.9226 - val_loss: 0.0350 - val_precision_1: 0.9815
Epoch 43/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0659 - precision_1:
0.9222 - val_loss: 0.0348 - val_precision_1: 0.9815
Epoch 44/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0656 - precision_1:
0.9224 - val_loss: 0.0345 - val_precision_1: 0.9815
Epoch 45/500
125/125 [==============================] - 1s 11ms/step - loss: 0.0652 - precision_
1: 0.9224 - val_loss: 0.0341 - val_precision_1: 0.9815
Epoch 46/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0650 - precision_1:
0.9251 - val_loss: 0.0338 - val_precision_1: 0.9815
Epoch 47/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0647 - precision_1:
0.9249 - val_loss: 0.0335 - val_precision_1: 0.9815
Epoch 48/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0644 - precision_1:
0.9254 - val_loss: 0.0333 - val_precision_1: 0.9818
Epoch 49/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0640 - precision_1:
0.9254 - val_loss: 0.0329 - val_precision_1: 0.9818
Epoch 50/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0637 - precision_1:
0.9256 - val_loss: 0.0327 - val_precision_1: 0.9818
Epoch 51/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0635 - precision_1:
0.9258 - val_loss: 0.0325 - val_precision_1: 0.9818
Epoch 52/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0632 - precision_1:
0.9256 - val_loss: 0.0323 - val_precision_1: 0.9818
Epoch 53/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0629 - precision_1:
0.9233 - val_loss: 0.0320 - val_precision_1: 0.9818
Epoch 54/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0627 - precision_1:
0.9231 - val_loss: 0.0318 - val_precision_1: 0.9818
Epoch 55/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0624 - precision_1:
0.9233 - val_loss: 0.0316 - val_precision_1: 0.9818
Epoch 56/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0621 - precision_1:
0.9231 - val_loss: 0.0314 - val_precision_1: 0.9818
```

```
Epoch 57/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0618 - precision_1:
0.9228 - val_loss: 0.0312 - val_precision_1: 0.9818
Epoch 58/500
125/125 [==============================] - 1s 7ms/step - loss: 0.0615 - precision_1:
0.9265 - val_loss: 0.0310 - val_precision_1: 0.9818
Epoch 59/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0613 - precision_1:
0.9240 - val_loss: 0.0308 - val_precision_1: 0.9818
Epoch 60/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0611 - precision_1:
0.9267 - val_loss: 0.0310 - val_precision_1: 0.9818
Epoch 61/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0608 - precision_1:
0.9271 - val_loss: 0.0306 - val_precision_1: 0.9818
Epoch 62/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0606 - precision_1:
0.9275 - val_loss: 0.0304 - val_precision_1: 0.9818
Epoch 63/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0604 - precision_1:
0.9275 - val_loss: 0.0303 - val_precision_1: 0.9818
Epoch 64/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0602 - precision_1:
0.9275 - val_loss: 0.0301 - val_precision_1: 0.9818
Epoch 65/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0600 - precision_1:
0.9275 - val_loss: 0.0304 - val_precision_1: 0.9818
Epoch 66/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0597 - precision_1:
0.9273 - val_loss: 0.0298 - val_precision_1: 0.9818
Epoch 67/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0596 - precision_1:
0.9273 - val_loss: 0.0301 - val_precision_1: 0.9818
Epoch 68/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0592 - precision_1:
0.9275 - val_loss: 0.0293 - val_precision_1: 0.9818
Epoch 69/500
125/125 [==============================] - 1s 7ms/step - loss: 0.0591 - precision_1:
0.9300 - val_loss: 0.0298 - val_precision_1: 0.9818
Epoch 70/500
125/125 [==============================] - 1s 8ms/step - loss: 0.0588 - precision_1:
0.9304 - val_loss: 0.0295 - val_precision_1: 0.9818
Epoch 71/500
125/125 [==============================] - 1s 9ms/step - loss: 0.0587 - precision_1:
0.9302 - val_loss: 0.0296 - val_precision_1: 0.9818
```

1.f

In [ ]:
```
model2.evaluate(test_ds)
```

```
 1/16 [>.............................] - ETA: 0s - loss: 0.0235 - precision_1: 1.000
0
16/16 [==============================] - 0s 7ms/step - loss: 0.0580 - precision_1:
1.0000
```

Out[ ]: [0.05803532898426056, 1.0]

In [ ]:
```
from sklearn.metrics import classification_report

y_pred = model2.predict(X_test)
predicted = tf.squeeze(y_pred)
predicted = np.array([1 if x >= 0.5 else 0 for x in predicted])

print(classification_report(y_test, predicted))
```

```
16/16 [==============================] - 0s 3ms/step
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       453
           1       1.00      0.83      0.91        47

    accuracy                           0.98       500
   macro avg       0.99      0.91      0.95       500
weighted avg       0.98      0.98      0.98       500
```

After changing the epoch count and retraining the model, we have gotten a precision of 1, recall of 0.83, and f1 score of 0.91. This meant that the model can predict the positive class with a precision of 100% and recall of 83% with an aggregate f1 score of 91%. This is a good result for the model.

In [ ]:
```
model2.save_weights('model2.h5')
```